ALGORITHMS AND PROBLEM SOLVING USING C++Internship Assignment Report

Submitted By
Sanjay Pushparaj
Bachelor of Technology – Artificial Intelligence and Machine Learning
SNS College of Technology, Coimbatore

1. Introduction

Algorithms form the foundation of computer science and software development. Every application, whether it is a web application, mobile application, operating system, or artificial intelligence system, relies on efficient algorithms for processing data and solving problems.

The purpose of this assignment is to implement and analyze commonly used algorithms in C++. The project focuses on understanding algorithmic approaches such as Divide and Conquer, Searching Techniques, and Dynamic Programming. Along with implementation, runtime measurement and complexity analysis were performed to evaluate the efficiency of each algorithm.

The algorithms selected for this project are:

1. Merge Sort
2. Binary Search
3. 0/1 Knapsack Problem

These algorithms represent different categories of problem-solving techniques and are widely used in practical software development and technical interviews.

2. Objectives

The main objectives of this assignment are:

- To understand algorithm design principles.
- To implement sorting algorithms for efficient data organization.
- To implement searching algorithms for quick data retrieval.
- To apply dynamic programming techniques to optimization problems.
- To analyze time and space complexity.
- To measure execution time using the Chrono Library in C++.
- To improve problem-solving and programming skills.

3. Development Environment

The project was developed using the following tools:

ComponentDescription

Programming Language

C++

Compiler

GNU G++

IDE

Visual Studio Code

Operating System

Windows 11

Version Control

GitHub

Compiler Information:

g++ (MinGW-W64) 15.2.0

The programs were executed through the Windows Command Prompt and tested with multiple input sizes.

4. Project Structure

The project is organized as follows:

```
Algorithms/
│
├──── MergeSort/
│     └──── mergesort.cpp
│
├──── BinarySearch/
│     └──── binarysearch.cpp
│
├──── Knapsack/
│     └──── knapsack.cpp
│
├──── screenshots/
│
└──── README.md
```

This modular structure improves readability and maintainability while allowing each algorithm to be developed independently.

5. Merge Sort

Overview

Merge Sort is a comparison-based sorting algorithm that follows the Divide and Conquer approach. It recursively divides an array into smaller subarrays until each subarray contains a single element. These smaller arrays are then merged back together in sorted order.

Merge Sort is particularly useful for large datasets because its time complexity remains consistent in all cases.

Working Principle

The algorithm performs the following steps:

1. Divide the array into two equal halves.
2. Recursively divide each half until a single element remains.
3. Merge the divided arrays while maintaining sorted order.
4. Continue merging until the complete array is sorted.

Advantages

- Efficient for large datasets.
- Stable sorting algorithm.
- Predictable performance.

Time Complexity

Case

Complexity

Best Case

O(n log n)

Average Case

O(n log n)

Worst Case

O(n log n)

Space Complexity

O(n)

Sample Input

7

38 27 43 3 9 82 10

Sample Output

Sorted Array:

3 9 10 27 38 43 82

Screenshot

Insert Merge Sort execution screenshot here.

6. Binary Search

Overview

Binary Search is an efficient searching algorithm used on sorted arrays. Instead of checking every element sequentially, the algorithm repeatedly divides the search space into halves.

This significantly reduces the number of comparisons required and makes Binary Search one of the fastest searching techniques available.

Working Principle

1. Find the middle element.
2. Compare it with the target value.
3. If equal, return the position.
4. If smaller, search the right half.
5. If larger, search the left half.
6. Repeat until the element is found or the search interval becomes empty.

Advantages

- Extremely fast for sorted data.
- Requires very few comparisons.
- Suitable for large datasets.

Time Complexity

Case

Complexity

Best Case

O(1)

Average Case

O(log n)

Worst Case

O(log n)

Space Complexity

O(1)

Sample Input

10 20 30 40 50 60

Search Element: 40

Sample Output

Element found at index 3

Screenshot

Insert Binary Search execution screenshot here.

7. 0/1 Knapsack Problem

Overview

The 0/1 Knapsack Problem is a classic optimization problem solved using Dynamic Programming.

The objective is to maximize the total profit obtained from selected items while ensuring that the total weight does not exceed the capacity of the knapsack.

Each item can either be selected completely or not selected at all, which is why it is called the 0/1 Knapsack Problem.

Working Principle

The algorithm constructs a Dynamic Programming table that stores optimal solutions for smaller subproblems.

For every item and capacity:

- Include the item if it improves profit.
- Exclude the item if it does not improve profit.
- Choose the maximum of both possibilities.

The final answer is obtained from the last cell of the DP table.

Advantages

- Guarantees optimal solution.
- Demonstrates dynamic programming concepts.
- Efficient for moderate-sized inputs.

Time Complexity

Case

Complexity

Dynamic Programming

O(nW)

Where:

- n = Number of items
- W = Capacity of knapsack

Space Complexity

O(nW)

Sample Input

Values:

60 100 120

Weights:

10 20 30

Capacity:
50
Sample Output
Maximum Profit = 220
Screenshot
Insert Knapsack execution screenshot here.

8. Compilation Commands

The programs were compiled using the GNU G++ compiler.
Merge Sort
g++ MergeSort\mergesort.cpp -o MergeSort\mergesort.exe
Binary Search
g++ BinarySearch\binarysearch.cpp -o BinarySearch\binarysearch.exe
Knapsack
g++ Knapsack\knapsack.cpp -o Knapsack\knapsack.exe

9. Execution Commands

Merge Sort
MergeSort\mergesort.exe
Binary Search
BinarySearch\binarysearch.exe
Knapsack
Knapsack\knapsack.exe

10. Runtime Analysis

The execution time for each algorithm was measured using the Chrono Library in C++.
The runtime analysis helps evaluate algorithm efficiency and scalability.
Merge Sort
Merge Sort demonstrated consistent O(n log n) performance even when the input size increased significantly.
Binary Search
Binary Search remained extremely efficient because the search space is reduced by half after each comparison.
Knapsack Problem
The Dynamic Programming approach successfully computed optimal solutions while maintaining acceptable execution time for moderate problem sizes.

The runtime results confirmed that algorithm selection has a significant impact on program performance.

10. Runtime Analysis

The execution time for each algorithm was measured using the Chrono Library in C++.

The runtime analysis helps evaluate algorithm efficiency and scalability.

Merge Sort

Merge Sort demonstrated consistent O(n log n) performance even when the input size increased significantly.

Binary Search

Binary Search remained extremely efficient because the search space is reduced by half after each comparison.

Knapsack Problem

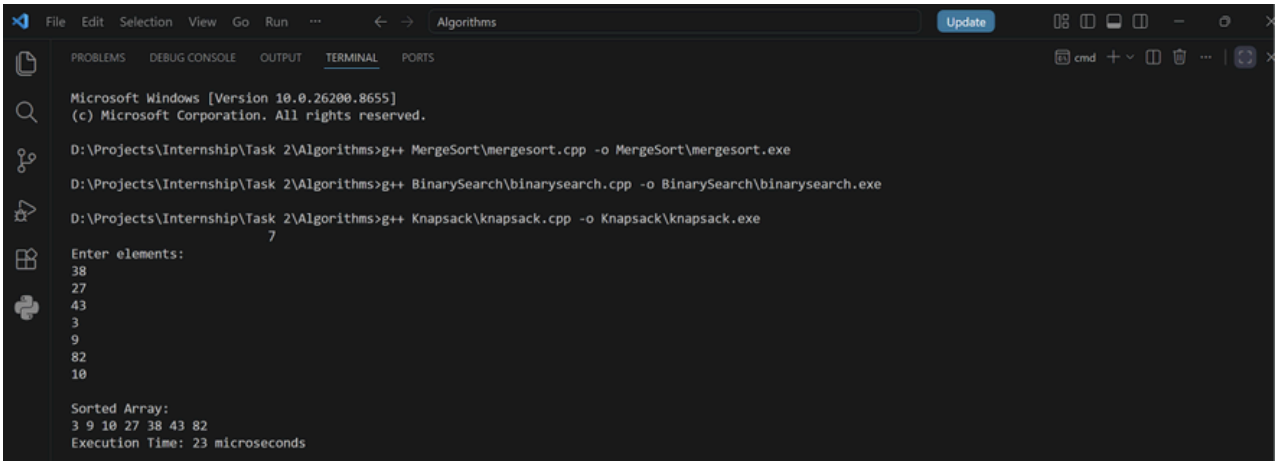
The Dynamic Programming approach successfully computed optimal solutions while maintaining acceptable execution time for moderate problem sizes.

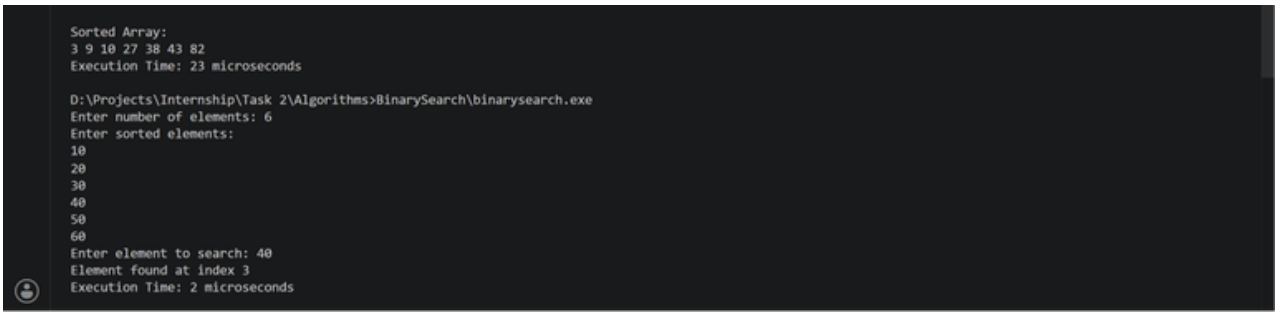
12. Conclusion

This project successfully implemented three important algorithms: Merge Sort, Binary Search, and the 0/1 Knapsack Problem. The implementation provided practical experience with different algorithmic paradigms and demonstrated how algorithm selection affects performance.

The assignment enhanced understanding of computational complexity, algorithm design, and optimization techniques. The knowledge gained through this project forms a strong foundation for advanced topics in data structures, software engineering, machine learning, and technical interview preparation.

Output


File Edit Selection View Go Run ···
Algorithms
Update
PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS
cmd
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.
D:\Projects\Internship\Task 2\Algorithms>g++ MergeSort\mergesort.cpp -o MergeSort\mergesort.exe
D:\Projects\Internship\Task 2\Algorithms>g++ BinarySearch\binarysearch.cpp -o BinarySearch\binarysearch.exe
D:\Projects\Internship\Task 2\Algorithms>g++ Knapsack\knapsack.cpp -o Knapsack\knapsack.exe
7
Enter elements:
38
27
43
3
9
82
10
Sorted Array:
3 9 10 27 38 43 82
Execution Time: 23 microseconds


Sorted Array:
3 9 10 27 38 43 82
Execution Time: 23 microseconds
D:\Projects\Internship\Task 2\Algorithms>BinarySearch\binarysearch.exe
Enter number of elements: 6
Enter sorted elements:
10
20
30
40
50
60
Enter element to search: 40
Element found at index 3
Execution Time: 2 microseconds


D:\Projects\Internship\Task 2\Algorithms>BinarySearch\binarysearch.exe
Enter number of elements: 6
Enter sorted elements:
10
20
30
40
50
60
Enter element to search: 40
Element found at index 3
Execution Time: 2 microseconds
D:\Projects\Internship\Task 2\Algorithms>Knapsack\knapsack.exe
Enter number of items: 3
Enter values:
60 100 120
Enter weights:
10 20 30
Enter knapsack capacity: 50
Maximum Profit = 220
Execution Time: 26 microseconds